

Informe del Trabajo Práctico - Procesamiento de imágenes 1

Integrantes: Lucía Jauregui - Norberto Narvaez - Nahir Sol Vera.

1° Problema: Ecualización Local del Histograma

El objetivo fue implementar una función que realice la ecualización local del histograma sobre una imagen en escala de grises, analizando un vecindario de tamaño $M \times N$ alrededor de cada píxel y ajustando su intensidad según la distribución local de intensidades.

La función implementada, **ecualizacion_local**, toma como parámetros la imagen original y el tamaño del vecindario. En esta función se aplica un filtro espacial no lineal, una transformación de intensidad basada en el histograma del vecindario.

```
def ecualizacion_local(img, M, N):
    filas, columnas = img.shape
    exp_y = M // 2
    exp_x = N // 2

    img_exp = cv2.copyMakeBorder(img, exp_y, exp_y, exp_x, exp_x, cv2.BORDER_REPLICATE)
    salida = np.zeros_like(img, dtype=np.uint8)

    for i in range(filas):
        for j in range(columnas):
            ventana = img_exp[i:i+M, j:j+N]

            hist, _ = np.histogram(ventana, bins=256, range=(0,256))

            hist = hist / (M*N)

            suma_acumulada = np.cumsum(hist)

            valor = img_exp[i+exp_y, j+exp_x]

            salida[i, j] = np.uint8(255 * suma_acumulada[valor])

    return salida
```

Dentro de cada vecindario se calcula el histograma de niveles de grises utilizando la función **histogram**, que representa las frecuencias absolutas de intensidades, es decir, cuántos píxeles tienen cada valor entre 0 y 255. Luego, el histograma se normaliza dividiendo cada frecuencia por el número total de píxeles del vecindario ($M \times N$), convirtiendo las frecuencias absolutas en frecuencias relativas.

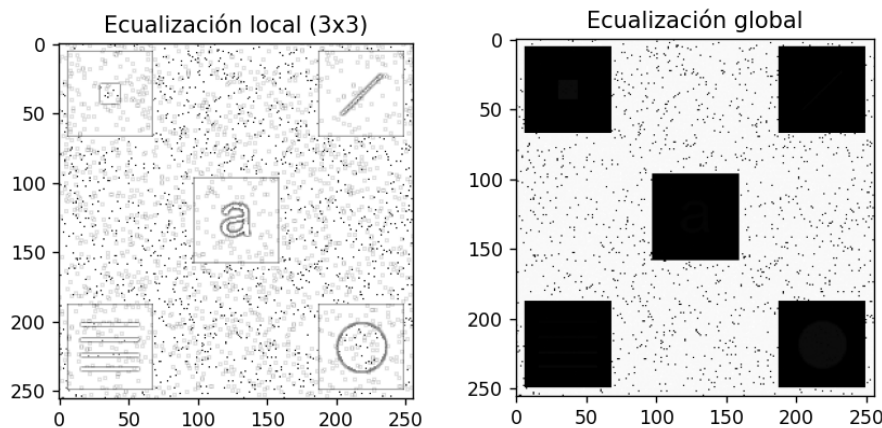
Se calcula la función acumulada del histograma normalizado, que mide la fracción de los píxeles del vecindario que tienen una intensidad menor o igual a un valor dado.

Finalmente, el nuevo valor del píxel central se obtiene a partir de esta suma acumulada, multiplicando el valor por 255 para escalarlo al rango $[0, 255]$. Cada píxel se reemplaza por su versión ecualizada localmente, ajustando el contraste en función de sus vecinos.

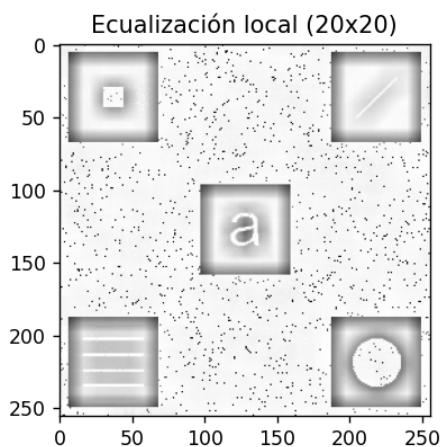
Durante la implementación, se observó que al analizar píxeles cercanos al borde de la imagen, parte del vecindario podía quedar fuera del área válida. Para resolver este problema se aplicó padding utilizando la función de OpenCV **copyMakeBorder**, que replica

los valores de los bordes. Esto asegura que todas las ventanas sean válidas, particularmente en las esquinas, evitando errores y permitiendo que la ecualización se aplique a toda la imagen.

Se aplicó la ecualización local a la imagen dada en el problema con distintos tamaños de ventana. En la ventana 3×3, pudimos ver que resalta detalles finos y mejora los bordes.



Probamos hacer la ecualización global para ver las diferencias y observamos que no se pueden observar los detalles de la imagen.



Conclusión:

La ecualización local mejora significativamente la visibilidad de detalles en regiones oscuras o iluminadas, adaptando el contraste de manera más precisa que la ecualización global. El uso del padding replicado garantiza una correcta ecualización incluso en los bordes. El tamaño de la ventana influye directamente en el resultado: ventanas pequeñas (3×3) realzan detalles locales, mientras que ventanas grandes (20×20) producen un efecto más general. Como resultado podemos ver que en los cuadrados negros de la imagen original después de aplicar la función realizada encontramos diferentes figuras (círculo, cuadrado, “a”, líneas).

2° Problema: Validación de formulario

El objetivo de este problema fue desarrollar un algoritmo capaz de validar formularios en formato de imagen. Cada formulario tiene varios campos (Nombre y Apellido, Edad, Mail, Legajo, Preguntas 1–3 y Comentarios), los cuales deben ser verificados según distintas condiciones dadas en el enunciado.

En primer lugar se realizó la binarización de la imagen mediante la función de openCV **adaptiveThreshold**, para resaltar las zonas escritas en blanco sobre fondo negro. A partir de esta imagen, se calcularon las sumas de píxeles por filas y columnas para detectar las líneas horizontales y verticales que delimitan las celdas del formulario. A partir de esto se aplicaron umbrales para aislar las líneas principales y luego se agruparon posiciones cercanas con la función **compactar_lineas**, obteniendo una única coordenada representativa por línea.

Para cada intersección de líneas se definieron los límites de una celda y se recortó su contenido mediante la función **extraer_celda_interior**.

Una vez aisladas las celdas se aplicaron distintas estrategias para la validación según el tipo de campo. Se aplicó la función **connectedComponentsWithStats** para contar las componentes conectadas y estimar la cantidad de caracteres. En las preguntas tipo *sí/no* se utilizaron contornos y áreas para detectar la presencia de marcas. Cada función de validación (**validar_nombre_apellido**, **validar_mail**, **validar_edad**, etc.) verificó las condiciones de longitud, cantidad de caracteres y presencia o ausencia de espacios, devolviendo “OK” o “MAL”. A su vez para la detección de espacio se desarrollaron dos funciones que detectan si hay un espacio significativo entre dos grupos de trazos.

Por otro lado, se detectó a partir del índice de la última componente conectada qué tipo de formulario encontrábamos, A, B o C.

Finalmente, se generó un archivo CSV (**resultados_formularios.csv**) con los resultados individuales de cada campo, y una imagen resumen (**resumen_resultados.png**) que permite visualizar qué personas completaron el formulario correctamente, en esta imagen se muestran los recortes del campo “Nombre y Apellido” de cada formulario, acompañados por un texto que indica si la persona “completó bien” o “completó mal” el formulario.

Algunos de los problemas que nos encontramos fueron:

- la detección de la columna que se encuentra entre el *sí/no*. Se nos ocurrió no tenerla en cuenta e implementamos una función **filtrar_lineas_verticales_por_longitud** que en los parámetros le ingresamos **fraccion_minima_long** que es la fracción mínima de la longitud máxima para conservar la columna. Con dicha función descartamos esa columna intermedia.

```
def filtrar_lineas_verticales_por_longitud(img_bin, columnas, fraccion_minima_long=0.8):
```

Las columnas antes de pasar por la función (4 columnas) :

Líneas detectadas (rojo=vertical, azul=horizontal)

		FORMULARIO A	
Nombre y apellido	JUAN PEREZ		
Edad	45		
Mail	JUAN_PEREZ@GMAIL.COM		
Legajo	P-3205/1		
	Si	No	
Pregunta 1	X		
Pregunta 2		X	
Pregunta 3	X		
Comentarios	ESTE ES MI COMENTARIO.		

Luego de aplicar la función la columna del medio ya no era parte de las columnas de la imagen, lo cual facilitaba la resolución para la parte de validación de las preguntas y el resto de las celdas.

Líneas detectadas (rojo=vertical, azul=horizontal)

		FORMULARIO A	
Nombre y apellido	JUAN PEREZ		
Edad	45		
Mail	JUAN_PEREZ@GMAIL.COM		
Legajo	P-3205/1		
	Si	No	
Pregunta 1	X		
Pregunta 2		X	
Pregunta 3	X		
Comentarios	ESTE ES MI COMENTARIO.		

Para detectar las marcas de las preguntas (1, 2 y 3) armamos esta función:

```
def contar_marcas_preguntas(celda, umbral_area=20):
```

La cual le pasamos la celda de la pregunta y el umbral a partir del cual detectamos una marca. Luego con la función **validar_pregunta_completa** validamos a partir de condicionales si la respuesta era correcta o no.

```
def validar_pregunta_completa(celda, numero, umbral_area=20):
```

Para detectar la marca fuimos cambiando el umbral de la función para que detectara la "X".

- Con el resto de las validaciones, cada campo requería un tratamiento distinto (palabras, números, marcas). Fue necesario ajustar los umbrales de área mínima y los rangos esperados de componentes conectadas para que el algoritmo funcione de manera correcta en cada caso. También estuvimos pensando cómo detectar los espacios entre cada palabra o número. Al principio no se podía distinguir correctamente entre caracteres separados (por ejemplo, "JUAN PEREZ" (Form. 1) o edad = "1 5" (Form.4). Lo cual pensamos en hacer estas dos funciones para poder resolver el inconveniente:

```
def cajas_componentes(celda_bin, area_min=35):
```

```
def hay_espacio(cajas, factor_espacio):
```

La primera obtiene las bounding boxes (cajas delimitadoras) de cada componente detectada, mientras que la segunda analiza las distancias horizontales entre dichas cajas. Si la distancia entre dos trazos consecutivos supera un cierto múltiplo del ancho medio de las letras (factor_espacio), se considera que existe un espacio significativo. Este factor fue ajustado manualmente según el tipo de campo, ya que se observó que un mismo umbral de área no servía para todos los campos. Por eso se definieron umbrales específicos para cada tipo.

Luego de la implementación de estas funciones pudimos ir validando cada campo con sus respectivas funciones.

- Otro desafío fue la parte B, la cual al principio no habíamos entendido que es lo que pedía la consigna, consistió en detectar que tipo de formulario era si A, B o C. En primer lugar hicimos un diccionario que dependiendo el formulario que era le asociamos el valor a su respectiva letra. {"formulario_01.png": "A"}.

Luego entendimos que había que reconocer la letra que se encontraba en el encabezado, por lo tanto lo resolvimos agarrando el índice de la última componente de la celda del encabezado, y a partir de su densidad determinamos que tipo de letra era, ya que B tiene más densidad que A, Y A es más densa que C. Otra forma de resolverlo también podría ser por los huecos que tiene cada letra.

Conclusión:

El algoritmo logró validar correctamente los formularios, detectando los campos y su contenido de manera automática. Aunque fue necesario ajustar varios umbrales y parámetros para que funcione bien dependiendo el tipo de dato. Los resultados de la validación son los siguientes:

JUAN PEREZ	completo bien
JORGE	completo mal
LUIS JUAN MONTU	completo bien
	completo mal
PEDRO JOSE GAUCHAT	completo mal